The Gate, Not the Predictor? Matched-Control and Endpoint-Metric Audits of Cache Prefetching and LLM Speculative Decoding

Simone Jarno Casartelli and Enrico Lopedoto
Algorithmica Solutions
simone@algorithmicasolutions.com, enrico@algorithmicasolutions.com

***Abstract*—Speculative systems predict future work and then decide which predictions to admit. Evaluations often change both components at once or substitute an admission-conditioned success ratio for the machine endpoint. We present two matched-control audits in different domains. In native ChampSim, a dominant-stride gate raises accepted-prefetch accuracy from 10.64% to 15.62% and suppresses 36.77% of L1D prefetches, yet changes DRAM reads by only -0.083% and loses 0.566% geometric-mean IPC. In a separately frozen Qwen2.5 speculative-decoding holdout, raising a draft-confidence threshold from 0.8 to 0.95 raises accepted-token fraction from 90.47% to 95.95%, yet reduces throughput by 10.37% on all eight prompts (exact sign-test $p = 0.0078125$). The less restrictive gate is 1.0627× target-only throughput; the higher-acceptance gate is 0.9525×. Both failures have the same cause: a ratio improves because its gate offers less work, while absolute useful work and downstream cost follow different paths. The contribution is a reproducible attribution and endpoint-audit method, not a new predictor, confidence gate, or universal performance claim.**

***Index Terms*—speculation, matched controls, cache prefetching, speculative decoding, performance methodology, negative results**

I. Introduction

Speculation is a recurring systems pattern. A cheap mechanism predicts work, a policy decides whether to admit it, and an expensive downstream machine executes or verifies it. Cache prefetchers predict addresses; LLM draft models predict tokens. Confidence filters suppress prefetches or end draft spans. Both domains commonly report a success ratio conditioned on admission: prefetch accuracy or draft-token acceptance.

That ratio is useful but not an endpoint. If a gate removes ten unsuccessful proposals and two successful ones, its success fraction rises while the amount of useful work falls. The downstream system may see no traffic reduction or latency benefit. Attribution is also lost when a gated learned predictor is compared with an ungated classical predictor: prediction and admission change together.

We ask whether these are domain-specific mistakes or one general experimental failure. Our contributions are:

- a shared propose–admit–execute decomposition and matched-control rule;
- a latency-aware local and native ChampSim audit that falsifies a broad neural-prefetching interpretation;
- a resource-bounded real-model LLM holdout that isolates only the draft confidence threshold and exhibits a predeclared acceptance/throughput reversal; and
- machine-verifiable artifacts that distinguish independent units from timing repetitions and reconstruct endpoints from raw runs.

The LLM result independently reproduces a warning present in recent speculative-decoding work. Novelty lies in the cross-domain attribution method and paired counterexamples, not in claiming that confidence-aware drafting is new.

II. A common audit model

Let P be proposed work, $A \subseteq P$ admitted work, and $S \subseteq A$ successful admitted work. A common proxy is

$$Q = |S|/|A|. \tag{1}$$

A stricter gate can raise Q while reducing both $|A|$ and $|S|$. Neither Q nor $|A|$ determines endpoint E, because admitted work may be absorbed, merged, arrive late, displace useful state, or cost more to generate than it saves.

We use three rules. First, compare gates with an identical predictor and predictors with an identical gate. Second, report absolute work at successive machine boundaries, not only the admission-conditioned ratio. Third, end at a user- or machine-visible metric: IPC and DRAM traffic for caches; generation latency and tokens/s for LLM inference.

Repeated timing of one deterministic workload estimates measurement noise but does not create new independent units. We therefore collapse repetitions within a trace or prompt before paired inference.

III. Case study I: cache prefetching

The original study compared a gated online 6–32–1 MLP delta predictor with an always-on stride predictor. It also obtained its largest speedups under instantaneous prefetch completion. We added classical predictors with the same admission policy, asynchronous latency, contention, finite MSHRs, an associative cache, and demand-only shadow state. Matched controls show no MLP advantage on random traffic and large MLP losses on several regular streams. The apparent safety came primarily from admission, not neural prediction.

TABLE I
THE SAME ENDPOINT-METRIC FAILURE IN TWO DOMAINS.

metric	looser/raw	stricter/gated
Prefetch accuracy	10.64%	15.62%
accepted L1D prefetches	100%	63.23%
DRAM reads	100%	99.917%
IPC ratio (strict/raw)	0.99434	
Draft acceptance	90.47%	95.95%
accepted draft tokens	1,623	1,278
geomean tokens/s	27.24	24.41
throughput ratio (strict/loose)	0.89628	

TABLE II
FROZEN LLM HOLDOUT; MEDIANS OVER THREE TIMINGS PER PROMPT.

prompt	gate .8	gate .95	.95/.8
GSM8K 10	27.06	25.75	0.9518
GSM8K 25	26.73	23.65	0.8848
GSM8K 50	22.80	21.06	0.9235
GSM8K 100	27.99	25.56	0.9130
HumanEval 10	24.47	21.67	0.8857
HumanEval 25	25.84	23.40	0.9057
HumanEval 50	32.39	26.77	0.8265
HumanEval 100	32.04	28.34	0.8845

A compact dominant-delta gate remained promising locally, so we froze a native holdout before observation: the highest-weight public DPC-3 SimPoint for each of nine previously unseen SPEC CPU2017 programs, with 50 million warm-up and 200 million measured instructions. A runtime ChampSim module selects no L1D prefetching, official `ip_stride`, a gate-disabled matched port, or gated stride from one binary. The disabled port is output-identical to official `ip_stride` on every recorded field.

Table I summarizes the result. The gate removes 36.77% of accepted L1D prefetches and raises accuracy by 4.98 percentage points, while retaining 92.83% of useful prefetches. Only 6.85% fewer prefetch misses reach LLC; DRAM reads barely change. Direct gated/raw geometric-mean IPC is 0.99434 (bootstrap 95% CI 0.98445–0.999997), with four wins and five losses. The proxy improvement does not establish bandwidth or performance benefit.

IV. CASE STUDY II: LLM SPECULATIVE DECODING

A. Question and controls

Speculative decoding uses a smaller draft model to propose tokens that a target model verifies in parallel, preserving the target distribution in the idealized algorithm [5], [6]. Static draft length can waste work when token difficulty changes. SpecDec++, dynamic assisted generation, and SVIP use learned or confidence/entropy signals to stop drafting adaptively [7], [8], [9].

This makes speculative decoding a direct test of the shared audit model. The draft model is the predictor, confidence is the admission/stopping gate, accepted-token fraction is the proxy, and decode throughput is the endpoint. We hold target, draft, quantization, prompt, output count, sampling, maximum draft length, runtime, and hardware fixed. The primary direct contrast changes only the minimum draft-token confidence from 0.8 to 0.95.

B. Frozen design

We use official Qwen2.5-3B-Instruct as target and Qwen2.5-0.5B-Instruct as draft, both Q4_K_M GGUF [13]. The CPU-only `llama.cpp` runtime is pinned to commit `5854625`. Runs use four threads pinned to four logical CPUs, one server slot, no prompt cache, greedy sampling, and exactly 96 generated tokens. The host is a Ryzen AI 9 HX PRO 370 with 24 GiB system memory. GPU execution is deliberately excluded.

Four HumanEval and four GSM8K prompts form untouched holdout units. Indices 10, 25, 50, and 100 in each public test set were frozen before observation. Three timing repetitions per prompt and policy are collapsed to their median. Configuration order is deterministically shuffled in each repeat block.

Development prompts at indices 0 and 5 exposed two facts before holdout. First, the 0.95 threshold gave higher acceptance but lower throughput than 0.8, forming the frozen confirmatory prediction. Second, verification batch shape changed some greedy Q4 outputs. We therefore forced equal output-token counts, retained output hashes as a diagnostic, and made no output-quality claim.

C. Holdout result

The 120-run matrix is complete. Raising the threshold removes 25.75% of draft proposals and 21.26% of accepted draft tokens. Acceptance fraction therefore rises by 5.48 percentage points even as absolute accepted work falls.

Throughput moves in the opposite direction. The paired geometric-mean ratio is 0.89628, and threshold 0.95 is slower on every prompt. The exact two-sided sign test gives $p = 0.0078125$ for the single predeclared contrast. Threshold 0.8 is $1.06274\times$ target-only throughput; threshold 0.95 is $0.95250\times$. Static lengths two and four are also slower than target-only at $0.94381\times$ and $0.93749\times$. A high accepted-token fraction is thus neither necessary nor sufficient evidence of speedup on this CPU.

Output hashes differ across policies for six of eight prompts, yet each policy's hash is stable across all three repetitions. Static length two matches target-only on every prompt, while other verification shapes sometimes select a different continuation. This is consistent with known finite-precision and batch-shape sensitivity of greedy inference [12]; it does not prove the kernel-level cause or a distributional violation. It does show that "lossless" should not be silently strengthened to byte-identical output for this quantized systems configuration.

V. CROSS-DOMAIN INTERPRETATION

The two cases are not merely metaphorical. Both gates condition the denominator of their headline metric. In caching, the gate suppresses 36.77% of admitted requests and accuracy

rises, but most removed work never reached DRAM or later returned as demand traffic. In LLM decoding, the stricter gate suppresses 25.75% of proposals and acceptance rises, but shorter verification groups lose more batching benefit than the saved draft work repays.

The result does not imply that gates are harmful. Gate 0.8 is the only tested LLM policy that improves aggregate throughput, and raw stride materially beats no-prefetch in the cache study. It implies that admission policy is a separate component whose cost must be evaluated at the endpoint.

VI. Related work

Classical and learned prefetchers span stride, confidence, spatial, and learned prediction [1], [2], [3], [4]. Our cache contribution is an attribution audit rather than a new state-of-the-art prefetcher.

Recent LLM work already recognizes the draft-cost/acceptance trade-off. Learning to Draft optimizes throughput rather than acceptance-length proxies with co-adaptive policies [10]. Ha et al. find acceptance rate to be a poor standalone predictor and draft cost to dominate throughput across self-speculative methods [11]. Our result is an independent CPU replication with a matched threshold and a cross-domain endpoint framework, not priority for that observation.

VII. Threats to validity

The cache holdout uses nine programs, one highest-weight SimPoint per program, one ChampSim configuration, and no multicore contention. The LLM holdout uses one consumer CPU, one quantized model pair, eight prompts from two datasets, batch size one, and fixed short outputs. The threshold was chosen on four development prompts. Neither case measures silicon energy or production serving load.

The LLM sign test has only eight units; its small p follows unanimous direction, not broad workload coverage. Timing repetitions are not counted as independent. Output divergence prevents a quality claim and may couple content with later draft difficulty even under equal token counts. Future work should add model families, Q8 or floating-point targets, GPUs, a second runtime, natural-language workloads, long contexts, per-round timing, and energy.

VIII. Reproducibility

The cache artifact contains 36 native runs and 21,240 local rows. The LLM artifact contains 120 raw runs. Manifests pin source, runtime, configuration, model, prompt, binary, and trace hashes. The verifiers reconstruct every saved aggregate and enforce the independent-unit and fixed-token invariants:

```
python -m benchmarks.verify_research
python champsim/verify_results.py \
  --results \
  results/champsim_holdout_standard
python \
  benchmarks/specdec_endpoint_audit.py \
  verify --output \
  results/specdec_holdout_qwen_cpu
```

IX. Conclusion

Across cache prefetching and LLM speculative decoding, a stricter gate improves an admission-conditioned success ratio while worsening the endpoint. The shared lesson is methodological: isolate prediction from admission, track absolute work through downstream boundaries, and end the argument at IPC, traffic, latency, or throughput. Proxy metrics explain mechanisms; they do not prove system benefit.

References

[1] T.-F. Chen and J.-L. Baer, "Effective hardware-based data prefetching for high-performance processors," *IEEE TC*, 1995.

[2] J. Kim et al., "Path confidence based lookahead prefetching," *MICRO*, 2016.

[3] M. Bakhshalipour et al., "Bingo spatial data prefetcher," *HPCA*, 2019.

[4] M. Hashemi et al., "Learning memory access patterns," *ICML*, 2018.

[5] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from Transformers via speculative decoding," *ICML*, 2023.

[6] C. Chen et al., "Accelerating large language model decoding with speculative sampling," arXiv:2302.01318, 2023.

[7] K. Huang, X. Guo, and M. Wang, "SpecDec++: Boosting speculative decoding via adaptive candidate lengths," *COLM*, 2025.

[8] J. Mamou et al., "Faster assisted generation with dynamic speculation," Hugging Face, 2024.

[9] Z. Zhang et al., "Draft model knows when to stop: Self-verification speculative decoding for long-form generation," *EMNLP*, 2025.

[10] J. Zhang et al., "Learning to draft: Adaptive speculative decoding with reinforcement learning," *ICLR*, 2026.

[11] J. Ha, K. Ganesan, A. Nguyen, T. Ahmed, and A. Moshovos, "Characterizing self-speculative decoding approaches for accelerating LLMs," *ICML AdaptFM Workshop*, 2026.

[12] J. Yuan et al., "Understanding and mitigating numerical sources of nondeterminism in LLM inference," *NeurIPS*, 2025.

[13] A. Yang et al., "Qwen2.5 technical report," arXiv:2412.15115, 2024.

[14] N. Gober et al., "The Championship Simulator: Architectural simulation for education and competition," arXiv:2210.14324, 2022.